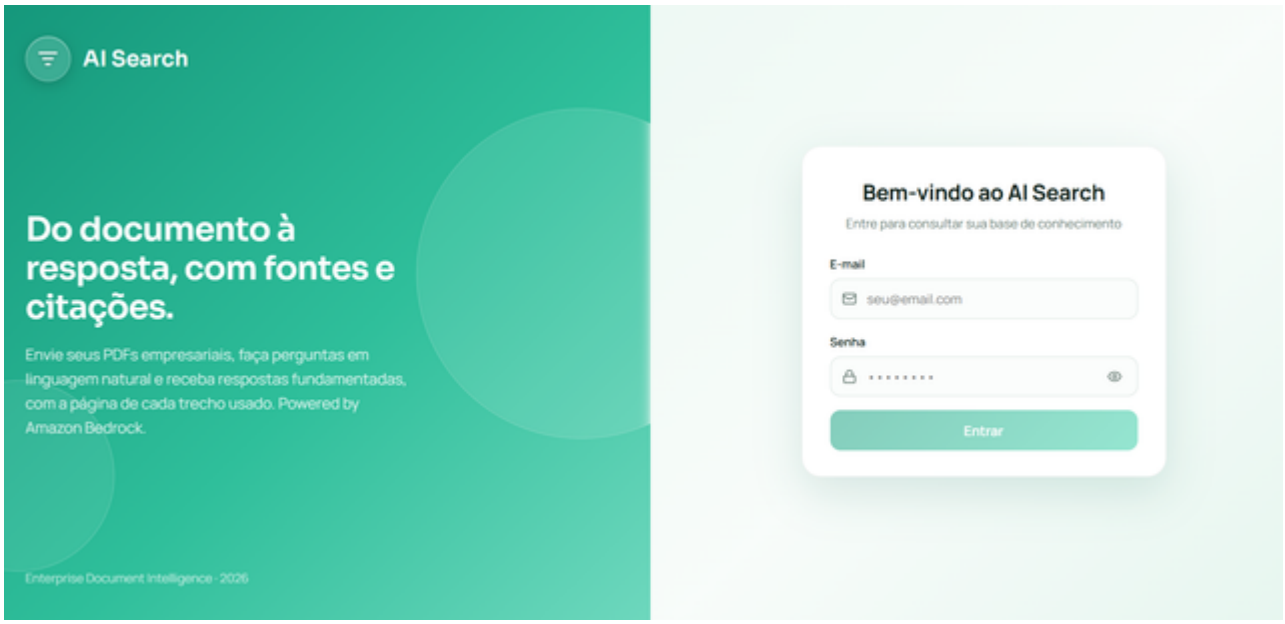
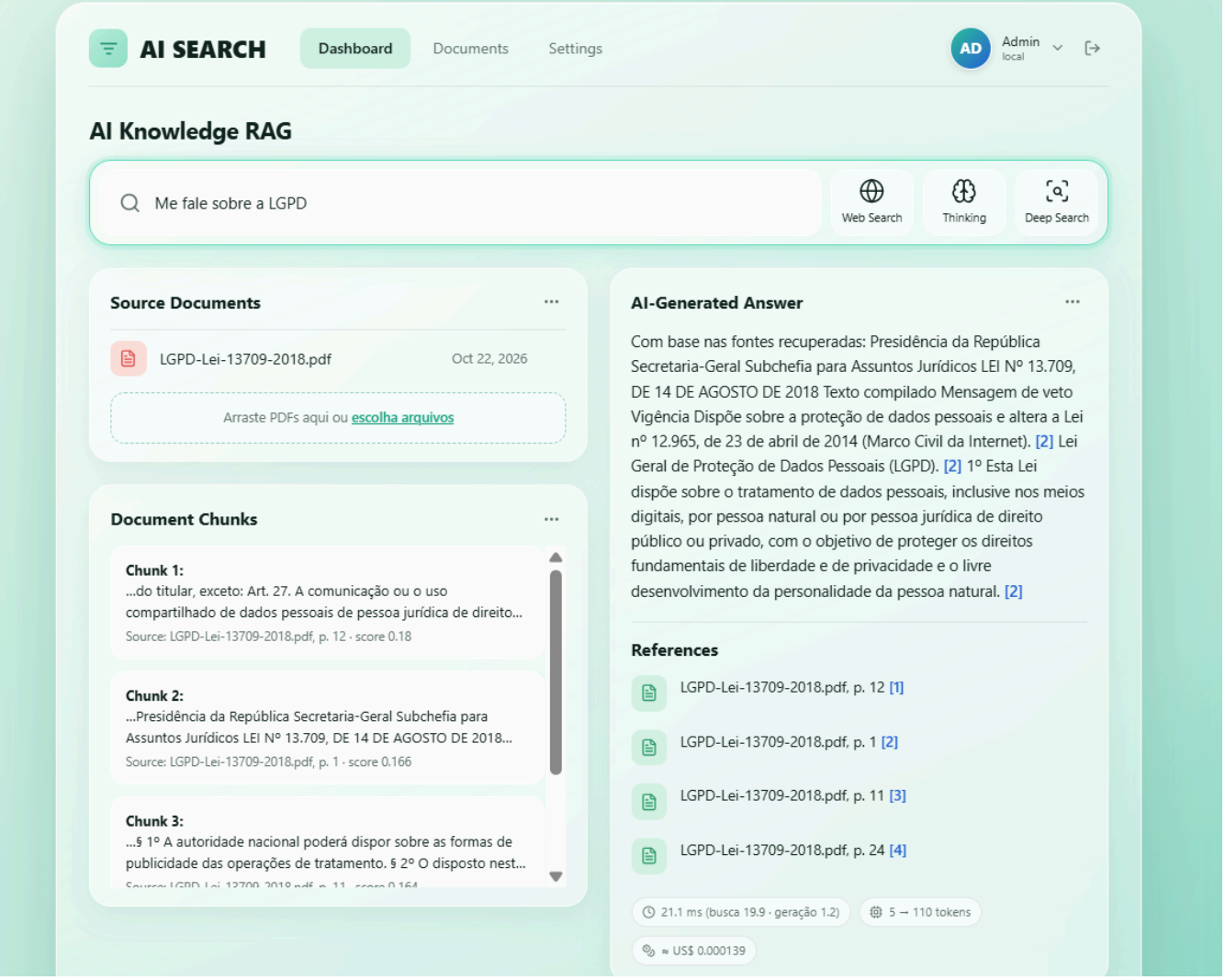
Document Intelligence API





RAG para documentos empresariais: envie PDFs, faça perguntas em linguagem natural e receba respostas **com citações** (documento + página). FastAPI + Amazon Bedrock, com métricas de latência e consumo de tokens por pergunta.



Rodar (sem Docker)

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

Abra http://127.0.0.1:8000 (interface) ou http://127.0.0.1:8000/docs (Swagger).

Frontend (React + Vite)

O código está em `frontend/`; o build (`app/static/`) já vem pronto e é servido pelo FastAPI, então **não precisa de Node para rodar**. Para mexer na interface:

```
cd frontend
npm install
npm run dev      # http://localhost:5173, com proxy para a API em :8000
npm run build    # gera app/static/
```

Providers

PROVIDER	O que faz	Custo
`local` (padrão)	Embeddings por hash + resposta extrativa. Demonstra o fluxo inteiro sem AWS.	zero
`bedrock`	Embeddings Titan v2 + geração via **Converse API** (Claude Haiku).	pago por uso

Para usar o Bedrock: copie `.env.example` para `.env`, defina `PROVIDER=bedrock`, configure as credenciais AWS (`aws configure`) e habilite o acesso aos modelos no console do Bedrock.

API

Método	Rota	Descrição
POST	`/documents`	Upload de PDF (indexa em trechos com página)
GET	`/documents`	Lista documentos indexados
DELETE	`/documents/{name}`	Remove documento
POST	`/ask`	`{"question": "...", "top_k": 4}` → resposta, `citations[]`, `metrics`
GET	`/health`	Status e provider ativo

Exemplo de `metrics` (também registrado em log a cada pergunta):

```
{"provider":"bedrock","retrieval_ms":212.4,"generation_ms":1480.2,"total_ms":1692.6,
 "input_tokens":812,"output_tokens":146,"estimated_cost_usd":0.000385}
```

Os preços ficam em `app/config.py` (`price_in_per_1k` / `price_out_per_1k`); confira a tabela vigente da AWS.

Decisões de arquitetura

- **RAG próprio em vez de Knowledge Bases / `RetrieveAndGenerate`:** esse caminho exige uma Knowledge Base, que costuma depender de OpenSearch Serverless (custo fixo

mensal). Aqui o índice é NumPy em disco (`data/`), e o código mostra cada etapa: extração → chunking → embeddings → busca por cosseno → prompt com trechos numerados → citações `[n]`.

- **Interface `Provider`:** trocar local ↔ Bedrock não muda o resto do código.
- **Prompt ancorado:** o modelo só pode usar os trechos fornecidos e deve citar `[n]`; sem base suficiente, diz que não encontrou.

Testes

```
.\.venv\Scripts\python.exe -m pytest
```

O provider Bedrock é testado com `botocore.Stubber` (sem chamar a AWS).

Docker (opcional)

```
docker build -t docintel . ; docker run -p 8000:8000 docintel
```

Limitações

PDFs escaneados (sem texto) não são suportados — seria preciso OCR (ex.: Amazon Textract). O índice em memória serve para centenas de documentos, não milhões.